



AI智能体实战：赋予大模型思考与行动的能力

深入解析两种核心认知框架：ReAct 与 Plan-and-Solve

什么是智能体（Agent）？一个拥有大脑和工具箱的LLM



解锁Agent潜能的关键：万物皆可为“工具”

通用工具（General Tools）



Search



Code Interpreter

执行Python、JavaScript代码



Database

与SQL/NoSQL数据库交互

专用及自定义工具（Specialized & Custom Tools）

可以集成任何内部系统或行业API，例如：



金融数据库

(Financial Databases)



企业内部知识库

(Internal Knowledge Base)



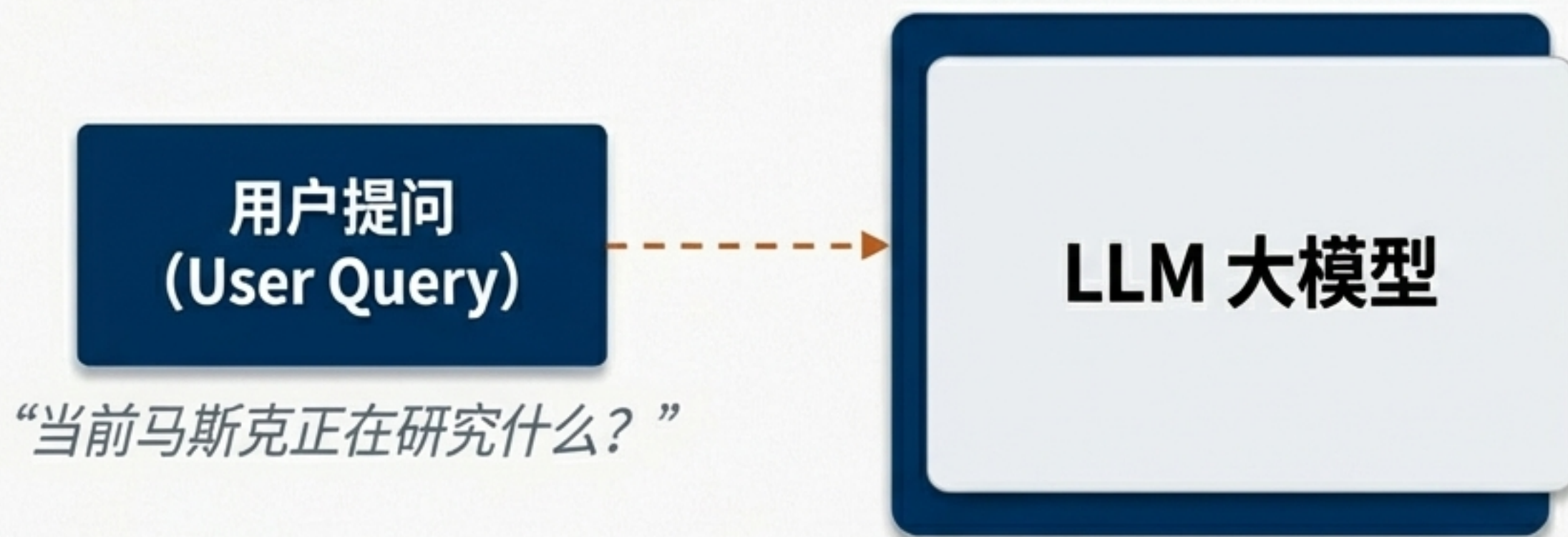
行业特定工具

(Industry-specific Tools)

通过丰富的工具，Agent能够超越LLM自身的知识边界，与真实世界进行交互和操作。

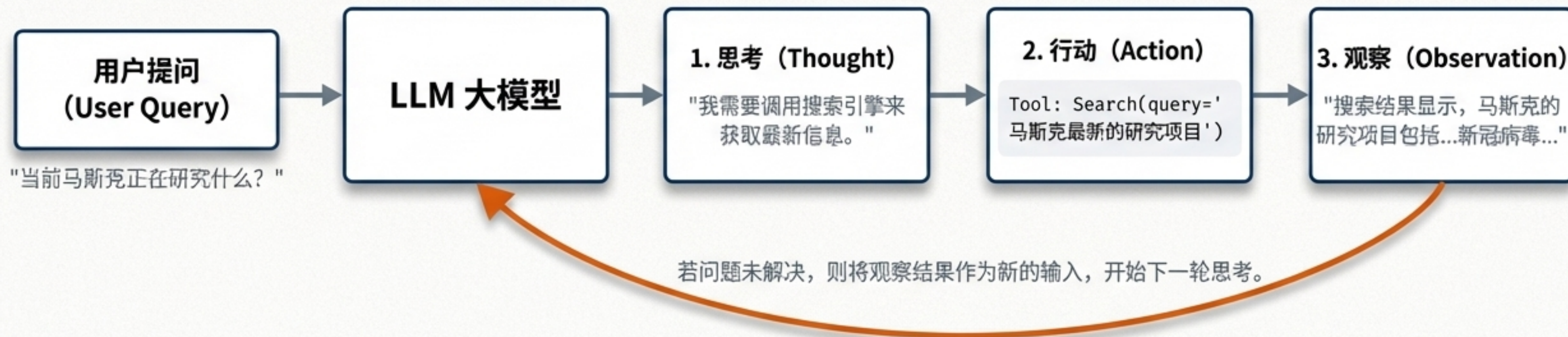
认知框架一：ReAct (Reason + Act)

像人一样思考：通过“思考-行动-观察”的循环解决问题



ReAct框架的核心是模仿人类的直觉式问题解决方法：先思考下一步该做什么，然后采取行动，观察结果，再根据结果调整下一步的思考。

ReAct框架的运作流程：一个持续的推理循环



这个循环可以重复N次, 直到LLM认为它已经找到了最终答案。这使得Agent能够处理信息不全的动态问题。

ReAct实战：用Prompt为大模型“编程”思考模式

```
template = """
```

尽你所能用中文回答以下问题。你可以访问以下工具：

```
{tools}
```

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of `[{tool_names}]`

Action Input: the input to the action

Observation: the result of the action

... (this Thought/Action/Observation can repeat N times)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

```
"""
```

在这里告诉LLM它拥有哪些可用的工具。

指令：强制LLM在行动前必须先输出思考过程。

指令：行动必须是调用一个已提供的工具。

占位符：用于将工具执行的结果反馈给LLM。

关键说明：明确告知LLM这是一个可以重复的循环结构。

ReAct实战：三步启动一个推理型Agent

核心代码

1. 加载工具

为Agent配备搜索引擎工具。

```
tools = load_tools(...)
```

2. 创建Agent

将LLM、工具和我们设计的Prompt组合在一起。

```
agent = create_react_agent(...)
```

3. 执行任务

输入问题，启动‘思考-行动’循环。

```
r = agent_executor.invoke("输入问题， ...")
```

运行实例

```
> Entering new AgentExecutor chain...
```

Thought: 这个问题涉及到当前科技领域的动态，我需要调用搜索引擎来获取相关信息。

Action: Search

Action Input: 马斯克正在研究的项目

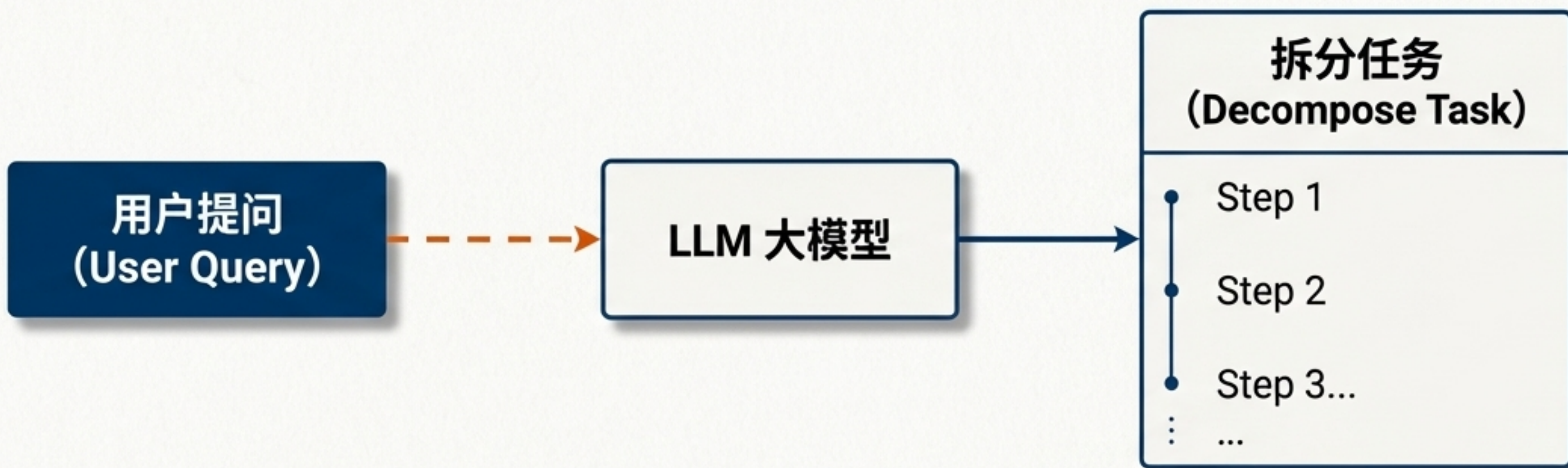
Observation: 这项研究旨在了解新冠病毒...

...

Final Answer: 马斯克目前的研究项目包括...

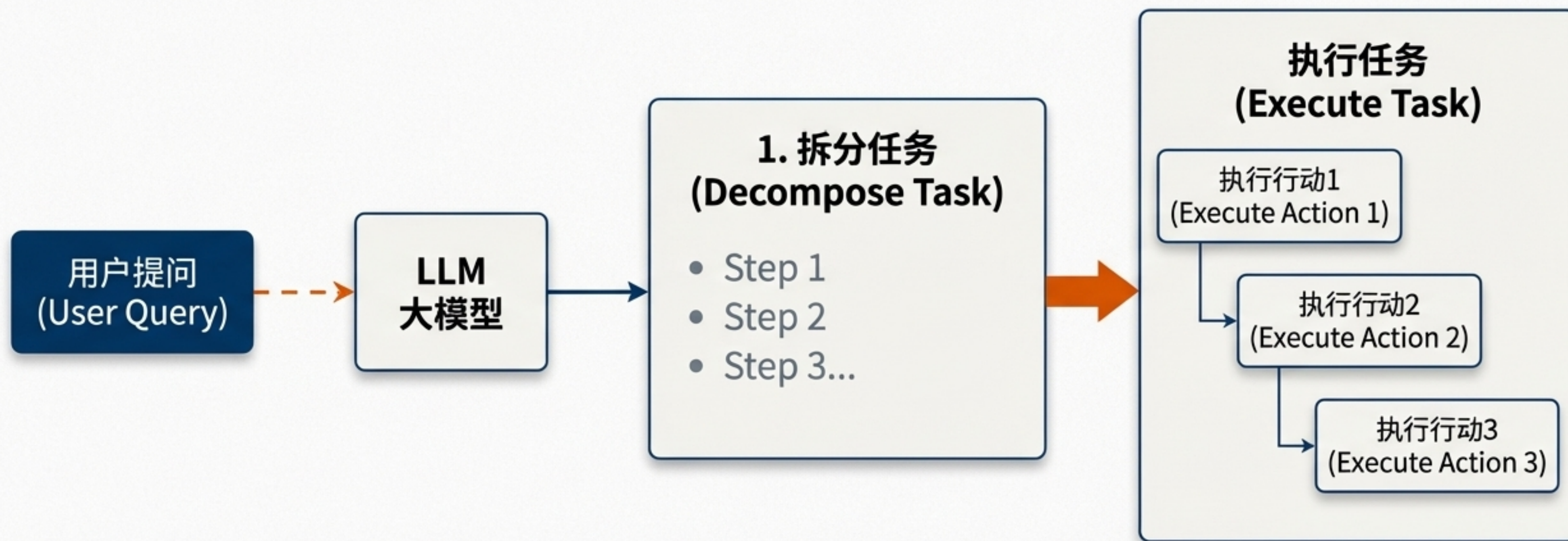
认知框架二：Plan-and-Solve

像战略家一样规划：先制定完整计划，再分步执行



与ReAct的即时反应不同，Plan-and-Solve模式首先要求LLM全面理解问题，并制定出一个详细的、多步骤的解决方案。这更适合需要复杂规划的任务。

Plan-and-Solve框架的运作流程：规划与执行分离



这种模式的优势在于规划的全局性。Agent在行动前就对整个解决方案有清晰的蓝图，减少了在执行过程中“走弯路”的可能性。

Plan-and-Solve实战：处理需要多步规划的复杂任务

左侧面板：核心代码

1. 定义工具集：提供库存查询、价格计算、配送安排等多个工具。

```
tools = [...]
```

2. 创建规划器与执行器：分别构建负责制定计划和执行计划的组件。

```
planner = load_chat_planner(llm)
executor = load_agent_executor(llm, tools)
```

3. 组合Agent：将规划器和执行器组合成一个完整的Plan-and-Solve Agent。

```
agent = PlanAndExecute(...)
```

4. 提交复杂任务：给出需要综合多个工具才能完成的任务。
示例任务："给出50台华为Mate60 Pro+的价格和配送方案"
`r = agent.invoke(...)`

右侧面板：运行实例

> Entering new PlanAndExecute chain...

```
steps=[
  Step(value='Research the current price...'),
  Step(value='Calculate the total cost...'),
  Step(value='check_inventory...'),
  ...
]
```

观察输出：Agent首先生成了一个清晰的、分步骤的行动计划，然后才开始逐一执行。

如何选择合适的认知框架？

Grounded Editorial

特性 (Feature)	ReAct (推理+行动)	Plan-and-Solve (规划+解决)
核心思想 (Philosophy)	迭代试错，边想边做 (Iterative trial-and-error)	全局规划，按部就班 (Holistic planning, step-by-step)
工作模式 (Mode)	动态循环 (Thought -> Act -> Observe)	线性序列 (Plan -> Execute Step 1 -> Execute Step 2...)
优势 (Strengths)	灵活，适应性强，能处理未知和动态变化的问题。	结构清晰，目标明确，适合可预测的、多步骤的复杂任务。
适用场景 (Use Cases)	- 实时信息查询 - 交互式问答 - 探索性数据分析	- 复杂的业务流程自动化 - 项目规划与执行 - 多工具协同任务

选择正确的框架，是构建高效、可靠AI智能体的第一步。

超越对话：智能体是大型模型走向通用人工智能的关键一步

传统的LLM擅长‘说’，而Agent赋予了它们‘做’的能力。

通过模拟人类的推理（ReAct）与规划（Plan-and-Solve）模式，
我们能够让大模型更智能、更可靠地解决真实世界的问题。

掌握Agent的构建方法，就是掌握了将AI能力从数字世界延伸到物理世界的钥匙。

